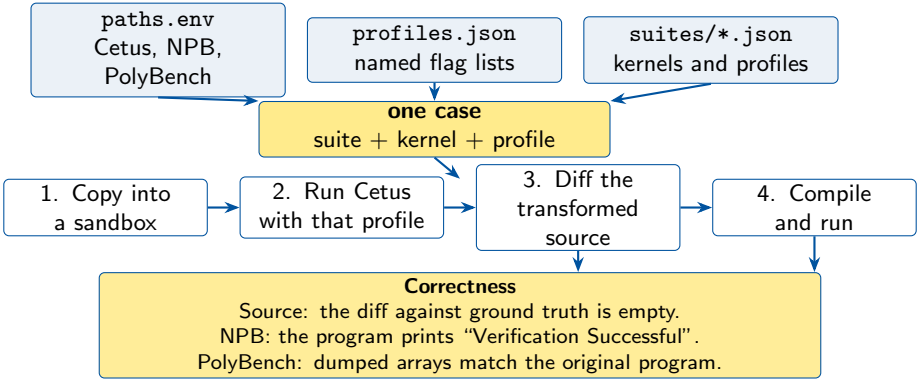
How a benchmark case is built



A **kernel** is one program (`gemm`, `bt`). A **profile** is one named list of Cetus flags (`parallel`, `paw_tiling_nt`). A **benchmark** is that program taken from PolyBench or NPB.

The JSON files in the diagram describe benchmark cases. Kernel snippets are JSON too, under `kernels/`. They stop after the source diff and are not compiled or run.

How a test is configured

A benchmark case is one triple from JSON: **suite**, **kernel**, **profile**. Kernel snippets are JSON suites too. Changing either file does not require rebuilding the harness.

1. Copy the kernel into a sandbox. The NPB and PolyBench trees stay untouched.
2. Run Cetus with that profile's flags.
3. Format the output and diff it against checked-in ground truth.
4. Compile the transformed program and run it.

Suite	Problem size	How the run is checked
NPB 3.3 serial	class S	4 threads, passive wait, verification string
PolyBench/C 4.2	MINI	1 thread, dumped arrays match the original
Kernel snippets	`kernels/*.json`	source diff only

Paths to Cetus, NPB, and PolyBench come from `paths.env`.

What is tested

Suite	Kernels	Count
PAW tiling snippets	GEMM, 3mm, and four nest-shape cases	6
Subscripted subscripts	AMG, EVSL, Gromacs, UA, and five others	9
NPB serial	BT, SP, LU, MG, CG, FT, EP, IS, UA	9
PolyBench	data mining, linear algebra, stencils, medley	30

Every profile starts from dependence analysis, privatization, reduction, and OpenMP output (`-ddt=2 -privatize=2 -reduction=2 -ompGen=1`).

Profile	Where	What is added
parallel	NPB and PolyBench	OpenMP only. No tiling.
paw_tiling_nt	NPB and PolyBench	PAW on. NT tile sizes, 4 cores, 25 600 KiB shared cache, 64-byte lines.
paw_tiling_lrw	NPB and PolyBench	PAW on. LRW (FindB) tile sizes. Same cache model.
paw_tiling_fixed32	NPB and PolyBench	PAW on. Every tiled loop uses size 32.
reuse_analysis	NPB only	Reuse-vector analysis on. No tiling.